

AI-ENG-110 · FINAL PROJECT · TOPIC 4

Async Research Assistant

A concurrent question-answering pipeline over Wikipedia, arXiv and web search, with cited synthesis and PostgreSQL-backed history.

Dilara Rzazada · Melek Askerova · Sanan Hajiyev · Nezrin Ibrahimova

19 SEPTEMBER 2026

Overview

01	Problem & Motivation
02	System Architecture
03	The AI Layer Contract
04	Concurrency Pipeline
05	Retry, Pacing & Timeouts
06	SE Layer Components
07	Performance Benchmark

08	Testing & Coverage
09	Robustness in Production
10	Caching & Storage
11	Docker & Configuration
12	Team Contributions
13	Limitations & Future Work
14	What We Delivered

The Cost of Sequential Retrieval

Research questions often need background material, academic papers and current web information. Retrieving those sources one after another introduces avoidable waiting and lets a single slow provider dominate the whole experience.

01

Sequential Latency

Three providers queried one after another compound each other's response time.

02

No Fault Isolation

A single failed or empty source must not crash the whole pipeline.

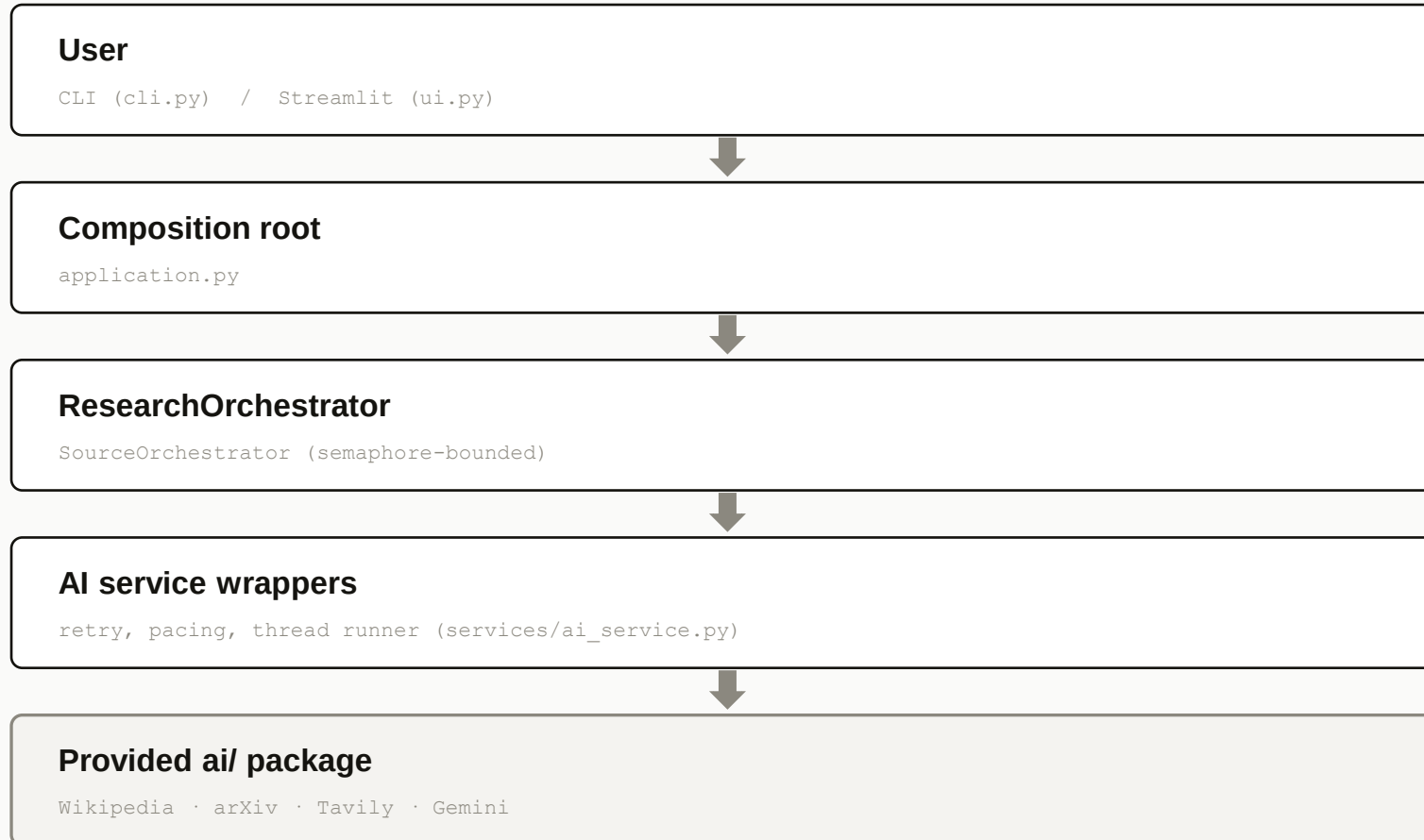
03

Repeated Provider Calls

The same question re-hits paid APIs on every request without a cache.

Our approach: `asyncio.gather` + a shared semaphore + TTL-aware PostgreSQL caching + graceful degradation

End-to-End Module Map



CROSS-CUTTING

Cache & History

PostgreSQL via asyncpg

Structured Logging

source, attempt, duration, request ID

Pydantic Validation

at every layer boundary

Source Aliasing

core/logic.py — pure formatting

Working Inside a Fixed Contract

01

Source Fetchers

fetch_wikipedia · fetch_arxiv · fetch_web share one `httpx.AsyncClient`, each returns `list[Source]`.

02

LLM Synthesizer

`synthesize(question, sources)` drops hallucinated indices and returns `AnswerWithCitations`.

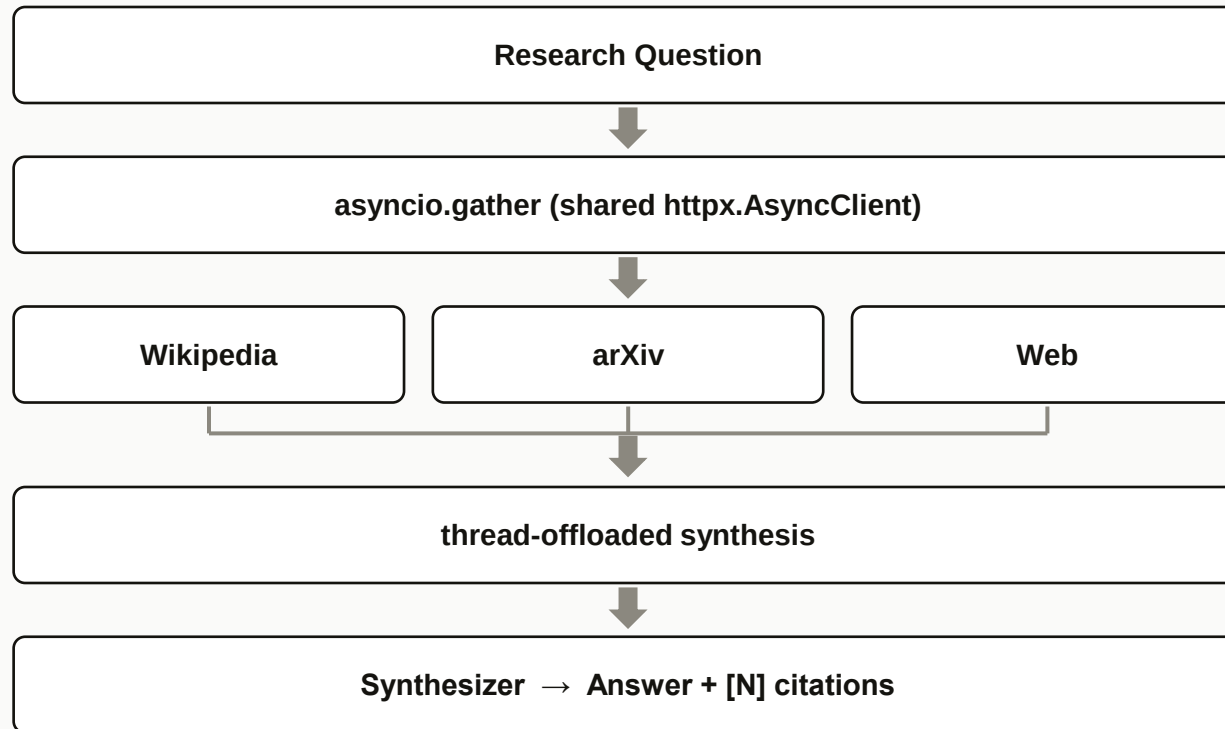
03

Provider Abstraction

Gemini selected for this run; Anthropic and OpenAI also work via env vars, with zero code changes.

Contract: `ai/` is read-only. A byte comparison of the reviewed repository's `ai/` files against the original Topic 4 package found no changes — the SE layer only calls `ai.sources.fetch_*`() and `ai.synthesizer.synthesize()`.

Fetching Three Sources at Once



Shared Semaphore

Three permits — enough to query all three sources for one question at once.

Graceful Degradation

A failed or empty source is reported, never lets it kill the whole answer.

Thread-Offloaded Synthesis

The synchronous LLM call runs on a dedicated `ThreadPoolExecutor`, one worker by default.

Wikipedia Transport Retry

Added at the transport level, since the supplied fetcher can silently skip a failed summary request.

Retry, Pacing and Timeout Budgets

AIFetchService

Source budget	10 s
Per-attempt deadline	2.5 s
Initial / max wait	0.25 s / 0.5 s
Max attempts	3

AISynthesisService

Attempt timeout	15 s
Initial / max wait	0.5 s / 6 s
Synthesis deadline	55 s
Total research deadline	75 s default

Pacing: a one-second minimum interval per source instance, including retries. Transient 429/5xx and transport errors get bounded retries with exponential jitter; identifiable permanent 4xx are not retried.

What the Team Built

`config.py`

Typed, env-driven pydantic-settings.

`models.py`

SourceOutcome, ResearchResult,
SessionRecord — Pydantic.

`storage/`

CacheStore ABC — InMemory and
Postgres implementations.

`services/ai_service.py`

Retry, pacing, thread runner around ai/.

`concurrency/`

ResearchOrchestrator +
SourceOrchestrator.

`cli.py`

ask / history commands, argparse-
based.

`ui.py` (Streamlit)

Bonus Ask + History views, in-process,
no HTTP.

Dockerfile + Compose

python:3.12-slim, non-root user,
PostgreSQL 16 service.

Sequential vs Concurrent

2.073x

Median concurrent-vs-sequential speedup across three repeats, five questions — 952.5 ms → 459.6 ms.

Repeat	Sequential	Concurrent	Ratio
1	939.1 ms	465.2 ms	2.019×
2	958.4 ms	459.6 ms	2.085×
3	952.5 ms	439.3 ms	2.168×

Offline, simulated collection only (Wikipedia 40 ms · arXiv 80 ms · web 40 ms) — not a live end-to-end measurement. Windows, Python 3.14.4, 20 Sept 2026, git 3f53954 (dirty).

364 Tests, 97% Coverage

364

TESTS PASSED

97%

STATEMENT + BRANCH · 98% LINE

0

NETWORK CALLS IN THE SUITE

MODULE	COV.
application	100%
cli	99%
ai_service	96%

MODULE	COV.
research	100%
orchestrator	99%
cache_store	96%

MODULE	COV.
retry	98%
transport	100%
ui	93%

Fakes, AsyncMock and HTTPX MockTransport throughout — including the original 16 supplied smoke tests. No paid requests required to run the suite.

Graceful Degradation on a Live Run

5/5

DEMO QUESTIONS COMPLETED

2

PROVIDERS REACHED, ONE EMPTY

9.58s

TOTAL, ANSWER CITED

REFERENCES

- [1] arxiv Retrospective of the ARPA-E ALPHA fusion program
- [4] web The Current State of Nuclear Fusion Research
- [5] web Nuclear fusion power, fusion energy research, ITER | IAEA

Sources with no results: wikipedia

Documents 6 Providers 2 Time 9.58s History: saved (session #13)

WHAT THE USER SEES

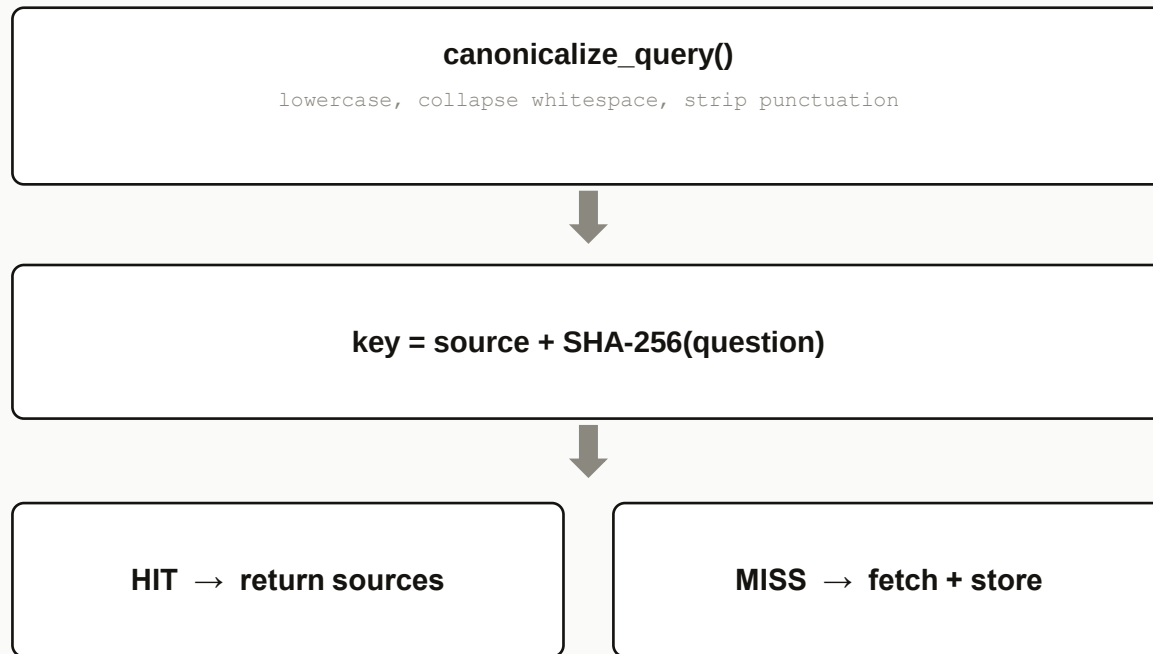
A cited answer from the providers that did respond, plus an explicit line naming the source that returned nothing.

WHAT THE CODE GUARANTEES

An empty source is a status, not an exception. Synthesis runs on the evidence available, and the session is still written to history.

One `source_client` factory builds every live client — HTTPS, redirects followed, host-specific Accept headers — so transport policy cannot drift between entry points.

TTL-Aware Source Cache



POSTGRESQL SCHEMA

cache_entries

key · payload JSONB · expires_at, TTL 86,400 s

research_sessions

question · answer · sources_used[] · sources_failed[] · duration_ms

Pool: lazy, lock-guarded, 1–5 connections. --no-cache bypasses both reads and writes.

Running It

DOCKER COMPOSE (PRODUCTION MODE)

```
docker compose up -d db
docker compose build app
docker compose run --rm app ask "..."
```

STANDALONE IMAGE

```
docker build -t researcher .
docker run --rm --env-file .env \
  researcher ask "..."
```

python:3.12-slim base, pinned requirements, non-root appuser, entrypoint python -m researcher. Compose starts PostgreSQL 16, waits on its healthcheck, mounts schema.sql and overrides the app DSN to the service hostname db.

VARIABLE

DEFAULT / BEHAVIOR

LLM_PROVIDER	gemini — Anthropic / OpenAI also supported
DATABASE_URL	required; Compose supplies its internal URL
CACHE_TTL_SECONDS	86,400 (24 h)

Who Built What

MEMBER	SHARE	KEY DELIVERABLES
Dilara Rzazada	25%	Config & data models; PostgreSQL setup, pool, cache and history storage; storage tests; final report
Melek Askerova	25%	AI fetch and synthesis services; retry and pacing; structured logging; Streamlit UI; service and UI tests; README
Sanan Hajiyev	25%	Concurrency and research orchestration; storage adapters; benchmark scripts; cancellation tests; shared HTTP client
Nezrin Ibrahimova	25%	CLI commands and input validation; application and Docker/Compose integration; demo scripts; CLI/integration tests
Shares reflect the team's agreed overall contribution — implementation, testing and documentation — not raw commit counts.		

What Still Needs Work

01

The 2.073× claim is narrow

It measures simulated collection only. A clean-revision, end-to-end, real-workload comparison remains outstanding.

02

Rate control has limits

Per-request semaphores aren't a shared account quota; a timed-out SDK thread can keep running.

03

Persistence needs strengthening

Citation and model snapshots, versioned cache keys, and startup DSN/range validation are not yet in place.

4. Release evidence is incomplete — no full prompt-injection defense, authentication, multi-user isolation, load testing, or a final measured image size.

What We Delivered

- ✓ `asyncio.gather` with per-source timeouts and a shared semaphore
- ✓ Graceful degradation — explicit partial-source notices
- ✓ PostgreSQL-backed TTL cache and session history
- ✓ CLI: `ask / history`, `--sources`, `--no-cache`
- ✓ Bonus Streamlit UI, sharing the same workflow in-process
- ✓ Exponential backoff, pacing and secret-safe structured logging
- ✓ Docker + Docker Compose with PostgreSQL 16
- ✓ 364 tests · 97% coverage

KEY ENGINEERING LESSONS

- ◆ `asyncio` is not a silver bullet — per-task timeouts matter as much as `gather` itself.
- ◆ Explicit service boundaries make the pipeline testable and diagnosable.
- ◆ Cache keys need canonicalization, or near-duplicate questions bypass the cache silently.
- ◆ Composition over inheritance keeps `ResearchOrchestrator` fully mockable offline.

TOPIC 4 · ASYNC RESEARCH ASSISTANT

Thank You

Questions & discussion

Dilara Rzazada · Melek Askerova · Sanan Hajiyeu · Nezrin Ibrahimova

`github.com/Dilara898/Software_Final-Project_M201`